Distilling and Deploying a Command-Parsing LM on a Local Cloud

CS3081 Cloud Computing - Mini Project Report

Nishant Suryavanshi | IIIT Manipur | Repository:
https://github.com/nish90033-lang/1012_Cloud-Assignment

Convention. Measured values are taken from experiment outputs in `results/`, training logs, Kubernetes observations, or CI runs. Assumptions and unmeasured values are identified explicitly.

1. Architecture

The project implements a command-parsing language model pipeline and deploys its inference service on a local Kubernetes cluster.

The teacher model, Phi-3.5, is served through Ollama on the Windows host laptop. The k3d cluster named `cmdparser` contains one server node and three namespaces:

- `data`: SeaweedFS, exposing an S3-compatible API and storing datasets and model artifacts.
- `pipeline`: two parallel teacher-labelling workers.
- `app`: the FastAPI inference service, Kubernetes Deployment, Service, Traefik Ingress and Horizontal Pod Autoscaler (HPA).

The data flow is:

`commands_pool.jsonl` → parallel teacher labelling → S3 teacher shards → cleaning and validation → student training → model artifacts → S3 and GitHub Releases → inference service.

The service downloads the selected model from SeaweedFS during startup. Credentials are supplied through Kubernetes Secrets generated locally and excluded from Git.

2. Task 1 - Object storage and parallel labelling

Setup. The cluster is created from `cluster/k3d.yaml`. SeaweedFS runs in the `data` namespace and exposes an S3-compatible interface. The teacher prompt in `prompts/teacher_prompt.txt` specifies the output schema, intent categories, slot-copying rules and examples. Ollama runs on the laptop, outside the Kubernetes cluster.

Sharding. The worker reads `SHARD_INDEX` and `SHARD_COUNT` and processes line numbers satisfying:

$$n \bmod \text{SHARD_COUNT} = \text{SHARD_INDEX}$$

The residue classes partition the input, so every command belongs to exactly one shard without coordination between workers. An Indexed Job uses two completions and parallel workers. Each worker writes its own object, uploads checkpoints every 25 records and resumes from its existing shard after a restart.

Both workers completed 1,500 records each, producing 3,000 teacher-labelled records. The Kubernetes Job completed 2/2 with no reported restarts. The measured wall-clock runtime was approximately 7 hours and 30 minutes. A controlled single-worker comparison was not performed, so a numerical parallel speed-up is not claimed. Both workers shared one Ollama instance on the same laptop, making teacher inference a compute bottleneck.

Cleaning. The original strict cleaner applies validation in order: invalid JSON, wrong keys, invalid intent, invalid slots, followed by duplicate removal.

Input	Invalid JSON	Wrong keys	Bad intent	Bad slot	Duplicates	Kept
3,000	2,137	2	1	24	9	827

The counts sum to 3,000. The strict-cleaner output checksum was:

`8c485e1f6fe5a724d86f8ffaff850c5689d3606558298d0040e85a5a9e412203`

Many teacher responses contained a valid JSON object followed by additional text. The strict cleaner rejected these responses because the complete output was not valid JSON.

A separate recovery pass used `JSONDecoder.raw_decode` to recover the first JSON object and then applied schema, intent, slot and duplicate checks. It recovered 2,821 records from the same 3,000 raw responses (93.6%). Its checksum was:

3dc8edb368d16516403ecfa5f501a95f8d48682085c3824977d676efcc65ba70

The recovery pass is documented separately and does not overwrite the original strict-cleaner output. The recovered dataset was used for the student training experiments.

3. Task 2 - Training the student

The student is EleutherAI Pythia-70M, fine-tuned using sequence-level distillation. Training uses next-token cross-entropy on command and teacher-response sequences, with loss applied to reply tokens only. The first 1,500 records from the recovered dataset were used.

Training configurations:

Parameter	v1	v2
Random seed	42	7
Learning rate	0.0001	0.0001
Batch size	16	16
Epochs	3	4
Maximum sequence length	128	128
Training records	1,500	1,500

Training loss:

Epoch	v1 loss	v2 loss
1	0.367825	0.360759
2	0.102909	0.107331
3	0.044520	0.059544
4	-	0.030795

Both experiments showed decreasing training loss. The resulting models were archived and published as GitHub Release assets.

Validation results. The evaluation used 100 validation examples. Metrics are percentages.

Model	JSON validity	Intent	Date	Time	Place	Full frame
Teacher Phi-3.5	27%	21%	20%	27%	26%	14%
Untrained Pythia-70M	0%	0%	0%	0%	0%	0%
Student v1	99%	84%	77%	89%	82%	46%
Student v2	100%	83%	77%	92%	81%	44%

Student v1 achieved the highest full-frame accuracy at 46%, compared with 44% for v2 and 14% for the teacher. V2 achieved 100% JSON validity and higher time-slot accuracy, but v1 performed slightly better on intent, place and full-frame accuracy. V1 was therefore selected for deployment.

The teacher's low JSON validity reflects its tendency to append additional text to generated JSON. The recovery pass salvaged valid initial JSON objects, allowing more examples to be used for training. The student learned to produce structured outputs more consistently than the teacher on this validation set. However, full-frame accuracy remained below 50%, showing that producing valid JSON does not guarantee that every extracted field is correct.

An observed model failure was the validation example V065, whose expected place was `quad cities`. The student generated a repetitive malformed place value instead. This caused schema validation to fail and the API to return `invalid_model_output`.

Sequence-level versus logit-based distillation. Sequence-level distillation trains the student on generated teacher outputs, using hard targets. Logit-based distillation instead trains the student to match the teacher's token-level probability distribution, typically using a divergence loss over the vocabulary.

Logit-based distillation was not implemented because Ollama supplied generated text rather than full-vocabulary logits. In addition, Phi-3.5 and Pythia use different tokenizers and vocabularies, preventing direct token-by-token distribution matching without an alignment strategy.

A limitation of the experiment is that the student sees only one generated answer per command. Some teacher mistakes can survive cleaning because the cleaner checks output structure and slot constraints rather than semantic correctness. The V065 example demonstrates that the student can still fail to copy a multiword location accurately.

4. Task 3 - The inference service

The inference service is implemented using FastAPI and listens on port 8000.

Endpoint	Behaviour
POST /parse	Returns the four-key JSON object and model-serving headers
GET /healthz	Reports whether the process is alive
GET /readyz	Returns 503 until the model has loaded

Successful parse responses include `X-Model-Version` and `X-Served-By`. The service supports configuration through `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` and `FAULT_INJECT`. Credentials are provided through a Kubernetes Secret.

Model downloading and loading run in a background thread. This allows the liveness endpoint to respond while readiness remains 503. Every model response is parsed and validated against the required schema. Invalid model output returns HTTP 500 with `{"error":"invalid_model_output"}`.

The Docker image uses Python 3.11.9 slim and CPU-only PyTorch dependencies, with pinned package versions. Its reported image content size was approximately 368 MB, below the 2 GB limit.

The automated service tests cover health and readiness, response headers, fault injection, malformed outputs and behaviour before model loading. The test suite completed with 21 passing tests and three deprecation warnings.

A separate 50-request sequential container test with explicit 1 CPU and 1 GiB limits, including a measured peak-memory result, was not recorded in the available experiment evidence. Kubernetes resource measurements are reported in Section 8 instead.

5. Task 4 - Kubernetes deployment, scaling and reliability

The `app` namespace contains the FastAPI Deployment, Service and Ingress. The Deployment uses two replicas during the two-replica test, with requests of 500m CPU and 512 MiB memory and limits of 1 CPU and 1 GiB memory. Liveness and readiness probes use `/healthz` and `/readyz`. RollingUpdate is configured with `maxSurge: 1` and `maxUnavailable: 0`.

The HPA is configured with a minimum of one replica, maximum of two replicas and a CPU utilisation target of 70% of the requested CPU. This corresponds to 350m CPU. The local cluster has one node, so this experiment does not demonstrate fault tolerance across separate machines. Metrics-server sampling also introduces a delay before scaling reacts.

Deployment and reliability checks:

Check	Observed result
Readiness while model loads	/readyz returned 503 during loading and 200 after the model became ready
Scaling	HPA scaled from one to two replicas under load and returned to one at low utilisation
Load balancing	20 successful requests were distributed 10/10 across the two original Pods
Rolling update v1 to v2	Deployment rollout completed successfully; API returned HTTP 200 with X-Model-Version: v2
Rollback	Rollback completed successfully; API returned HTTP 200 with X-Model-Version: v1
Pod deletion and recovery	Deleted Pod was recreated; replacement became Ready while the other replica remained Running
Cold start	8 s, 7 s and 8 s; median 8 s

The rolling-update load test is discussed in Section 8. It recorded 17 failed requests out of 1,151, or 1.48%. This exceeds the assignment's 0.5% rolling-update failure threshold. The failures included both HTTP 502 responses and invalid model outputs.

The Pod-recovery test was performed by deliberately deleting one application Pod while the second replica remained healthy. The replacement Pod reached Ready approximately 12 seconds after the first observation. This confirms controller-based recovery, but a separate request-level failure count during the deletion test was not collected.

Cold-start times measure Pod creation-to-Ready using three runs with one desired replica. They are not a pure model-loading benchmark. The startup logs confirmed that model files were downloaded from SeaweedFS.

6. Task 5 - Subnetting and NACL

The address space starts at `10.20.0.0`. Subnets were allocated from largest to smallest.

Subnet	Network/prefix	Usable range	Broadcast	Usable hosts
app (1,000)	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The complete calculations are in network/subnets.md.

The NACL in network/nacl.json is stateless. Its inbound rules deny 198.51.100.0/24, allow TCP ports 80 and 443, allow SSH from the management subnet and allow ephemeral reply traffic from the application subnet.

Outbound rules allow TCP port 8000 to the application subnet and ephemeral TCP ports 1024-65535 to clients for replies.

The simulator command:

python tools/nacl_sim.py --check network/nacl.json network/packets_sample.jsonl

produced **6/6 passing test cases**.

A NACL is stateless, so return traffic requires its own allow rule. Rules are evaluated in ascending rule-number order, making it necessary to place the deny rule before the broader HTTP allow rules.

One design limitation is that the specified ephemeral range includes port 8000. Consequently, the outbound ephemeral-port rule also permits traffic to port 8000 outside the application subnet. A stateless rule cannot distinguish a reply from a new connection based only on that port range.

7. Task 6 - Reproducibility, testing and CI/CD

The repository includes a Makefile for environment setup, training, publishing and Kubernetes operations. The Kubernetes manifests and scripts are version-controlled. The GitHub Actions workflow ci.yml performs linting, secret scanning, unit tests, model evaluation and a quality gate.

The automated test suite completed with **21 passing tests**. GitHub Actions passed for the main branch and both model releases:

- Main branch: https://github.com/nish90033-lang/1012_Cloud-Assignment/actions/runs/42334110400
- Model v1: https://github.com/nish90033-lang/1012_Cloud-Assignment/actions/runs/42334266817
- Model v2: https://github.com/nish90033-lang/1012_Cloud-Assignment/actions/runs/42334481000

The quality gate uses a minimum JSON-validity threshold of 95%. The untrained baseline scored 0% JSON validity, while the trained v1 and v2 models scored 99% and 100%, respectively. The untrained model therefore fails the threshold and both trained models pass it.

The required untrained-baseline failure demonstration was executed through GitHub Actions. Lint and Gitleaks, unit tests, and model evaluation passed; the quality-gate job then failed as expected because the untrained model did not meet the 95% JSON-validity threshold. Baseline failure run.

The initial main-branch CI run encountered a release-publication timing race: the workflow attempted to download the v1 release asset before it had finished publishing. Re-running the workflow after publication succeeded.

The secret scan reported no findings in the reviewed repository contents. Generated credentials and model artifacts are excluded from Git. The repository contains AI_USE.md, which documents AI assistance and validation.

Repository: https://github.com/nish90033-lang/1012_Cloud-Assignment

8. Task 7 - Performance, cost, SLO and review

Load testing. The service was tested with Locust using four concurrent users. The two-minute tests and the longer rolling-update test produced the following measurements.

Test	Requests	Failures	Error rate	Average latency	p50	p95	Throughput
Load test 1	370	4	1.08%	1,285 ms	1,600 ms	2,100 ms	3.09 req/s
Load test 2	442	4	0.90%	1,078 ms	850 ms	1,800 ms	3.70 req/s
Rolling update	1,151	17	1.48%	1,248.70 ms	1,200 ms	2,200 ms	3.20 req/s

In the rolling-update test, eight failures were HTTP 502 Bad Gateway responses and nine were HTTP 500 invalid_model_output responses. The 502 responses occurred during the rolling-update test and indicate temporary upstream availability problems. The 500 responses were caused by malformed model output, including the reproducible V065 location-generation failure.

Resource utilisation. During the rolling test, sampled Pod CPU usage reached approximately 1,000m, or one full CPU core. Observed memory reached approximately 611 MiB on the sampled Pods. In the earlier load test, sampled peak memory was approximately 679 MiB on one Pod and 593 MiB on the other. CPU saturation suggests that inference is CPU-bound under concurrent requests.

Cold start and recovery. Three Pod creation-to-Ready measurements were 8, 7 and 8 seconds, with a median of 8 seconds. During the recovery test, Kubernetes recreated a deliberately deleted Pod. The replacement became Ready approximately 12 seconds after the first observation, while the second replica remained available.

Cost estimate. The supplied rate card specifies ₹3.50 per vCPU-hour and ₹0.45 per GiB-hour. Assuming two continuously billed replicas, each with limits of 1 vCPU and 1 GiB:

- Per-replica hourly cost = 3.50 + 0.45 = **₹3.95**
- Two-replica hourly cost = **₹7.90**
- Measured rolling-test throughput = **3.1996 requests/sec**
- Estimated requests per hour = **11,518.52**
- Estimated cost per 1,000 requests = **₹0.686**

This is an estimate based on resource limits and two replicas. It excludes other infrastructure charges and temporary rollout surge capacity.

SLO and error budget. The target is 99% successful requests per load-test run. For the rolling-update test, 1,151 requests were made and 17 failed.

- Availability = (1,151 - 17) / 1,151 × 100 = **98.523%**
- Error budget = 1% × 1,151 = **11.51 requests**
- Failures = **17**
- Error budget consumed = 17 / 11.51 × 100 = **147.70%**
- Result: **SLO missed**

The rolling-update test also exceeded the specified 0.5% rolling-update failure threshold. This result is retained rather than excluded from the analysis.

Component	Evidence (measured)	Limitation found	Improvement
Data and storage	3,000 raw records; 827 strict-cleaned records; 2,821 recovered records; recovery dataset checksum recorded	Single-pod SeaweedFS without demonstrated volume replication	Replicate volumes and version datasets
Model quality	v1 full-frame accuracy 46%; v2 44%; teacher 14%	1,500 training examples; some teacher errors survive structural cleaning	Collect more data, filter using semantic agreement and explore logit distillation with vocabulary alignment
Latency and throughput	Rolling test p50 1,200 ms, p95 2,200 ms, 3.20 req/s	CPU-bound inference and batch size 1	Dynamic batching, quantisation and ONNX evaluation
Resource utilisation	Sampled CPU up to approximately 1,000m; memory up to approximately 611 MiB in rolling-test samples	CPU approaches the configured limit under load	Tune requests and limits using sustained measurements; evaluate VPA
Reliability and recovery	Rolling update 98.52% success; replacement Pod Ready in approximately 12 seconds; cold-start median 8 seconds	Single-node cluster; rollout had 502 errors	Multi-node deployment, improved readiness and traffic-draining behaviour, model caching
Security and cost	Gitleaks reported no findings; estimated cost ₹0.686 per 1,000 requests	HTTP within the cluster and one shared S3 identity	TLS, per-service credentials and NetworkPolicies

Overall review. The project demonstrates an end-to-end pipeline from teacher-generated labels and object storage to student fine-tuning, model publication and Kubernetes inference. The student substantially improves structured-output validity over the untrained baseline and the evaluated teacher outputs. The service successfully supports health checks, readiness, model-version headers, rolling updates, rollback and Kubernetes self-recovery.

The main remaining limitations are model field accuracy, CPU saturation and transient failures during rolling updates. The invalid-output errors are a model-quality problem, while the 502 responses are an availability problem. They require separate improvements. A larger and more diverse dataset, stronger output constraints, CPU-efficient inference and more robust traffic draining are appropriate next steps.

References

1. Kim and Rush, “Sequence-Level Knowledge Distillation,” EMNLP 2016.
2. Bastianelli et al., SLURP, EMNLP 2020 (CC BY 4.0).
3. EleutherAI, Pythia-70M model card.
4. Project source code, configuration, experiment logs and evaluation outputs: https://github.com/nish90033-lang/1012_Cloud-Assignment